

Date _____
Page No. _____

Name: Manu Sharan Kumar

Roll No: ACE080BC7037

Subject: Software Engineering.

8.1. "Software is developed, not manufactured."
Explain this statement. How does it affect software quality compared to hardware manufacturing?

Ans Hardware manufacturing is a physical, repetitive process where raw materials are assembled into identical physical components based on a predefined design. Software development, on the other hand, is an intellectual, creative process. Every piece of software is custom-built through logical thinking, problem-solving and coding.

Also, in hardware, creating a second copy requires the same amount of materials and labor. In software, once the "master" copy is developed, replication (copying the software) costs virtually nothing.

Ans Hardware defects usually occur due to manufacturing flaws (e.g. bad materials, physical tolerance) or wear-and-tear over time. Software defects are exclusively design and logical flaws introduced during development that were undetected during testing.

Hardware Quality Control (QC) focuses on physical inspection and testing against physical tolerance. Software QC requires rigorous logical verification, edge-case testing, and code reviews because a

single logical error can break the entire system. Hardware quality declines over time due to environmental stress and physical friction. Software quality does not degrade due to physical wear, but rather due to "software rot" - i.e. the accumulation of bugs and side effects when the code is modified over time to meet changing the requirements.

2. Explain how modern software engineering practices helped overcome the software crisis. Justify this statement: "Software doesn't wear out like hardware components."

Ans The software crisis (1960s-1980s) was characterized by projects going over budget, missing deadlines, delivering low-quality software, and being unmaintainable. Modern software engineering practices introduced discipline to overcome this:

- **Structured Process Models**
Moving away from the "Build and Fix" approach to systematic models (like Agile, Scrum, and DevOps) ensured predictable timelines and continuous delivery.
- **Rigorous Requirements Engineering**
Explicitly gathering, analyzing, and documenting requirements minimized misunderstandings between clients and developers.
- **Automated Testing & CI/CD**
Continuous Integration and Continuous Deployment

CC/CD pipelines catch bugs early, ensuring high software quality before deployment.

- Object-Oriented & Modular Design

Designing software as reusable, independent components makes large-scale codebases easier to maintain and scale.

Ans

Physical hardware experiences high failure rates early on (burn-in phase), settles into a stable low-failure period, and then wears out due to friction, heat and physical degradation (wear-out phase).

Software is immune to environmental and physical friction. A line of code will execute exactly the same way a million times. However, software exhibits an "idealized" flat failure rate after initial bugs are ironed out. In reality, the failure rate spikes everytime software is modified or updated to accommodate new requirements. This is due to regression bugs, not physical wear-and-tear.

3. A banking system has been running smoothly for 10 years without any major changes. The bank manager says, "Our software is perfect because it never crashes." As a software engineer, how would you respond to this statement?

Ans

As a software engineer, I would politely disagree with the claim that the software is perfect. A system that hasn't crashed in 10 years proves

reliability under static conditions, but it does not equate to perfection or future viability.

A software system used in a real-world environment must continuously change to remain useful; otherwise, it becomes progressively less useful. Over 10 years, banking regulations, customer expectations, and security change drastically. While, the software may not crash, it is highly susceptible to modern cyberattacks, lacks encryption standards, and contains unpatched legacy vulnerabilities. Modern banking requires seamless integration with APIs, mobile apps, and cloud infrastructure. A 10-year-old static system creates a bottleneck for digital transformation. Because no major changes were made, the current team likely lacks deep knowledge of the legacy system. If a catastrophic failure finally occurs, fixing it will be incredibly difficult and expensive.

4. A project manager says, "We don't need software engineering. Our developers are excellent programmers. We will just write code and make it work."

~~Q~~ What arguments would you give to convince him that software engineering is necessary?

Ans I would present the following arguments to demonstrate that programming is only a small subset of software engineering

- Programming is construction; Engineering is the Blueprint

Excellent programmers write great code, but without software engineering, they lack architecture. Without proper architectural design, the system will eventually become an unmanageable, spaghetti-coded monolith that cannot scale.

- Cost of Change Increases Exponentially:

Without engineering practices like formal requirements gathering, errors will be caught late in the development cycle or after release. Fixing a requirement error during development "costs a fraction of what it costs to fix it after the code has been written and deployed."

- Individual Capability vs Team Coordination

Excellent programmers working in isolation will build disjointed components. Software engineering provides the "umbrella activities" (Configuration Management, Risk Management, and Project Tracking) required to sync a team's efforts.

- Maintainability and Documentation

Software is often maintained by developers who didn't write the original code. Software engineering mandates clean documentation, design patterns, and readable code structure, ensuring the company isn't entirely dependent on a few specific individuals.

5. Consider two projects: (1) A heart pacemaker control system, (2) A mobile game. Would you use the same software development approach

for both? Justify your answer with reasons.
Ans No, I would absolutely not use the same software development approach for both projects. They sit at opposite ends of the software characteristics spectrum.

	Project 1	Project 2
System Category	Safety-critical or Embedded System	Consumer or Entertainment System
Tolerance for Failure	Zero. A bug means physical injury or death.	High. A crash means an annoying app restart.
Requirements	Fixed, rigid, and strictly bounded by medical standards.	Dynamic, changing based on user feedback and trends.
Recommended Model	Waterfall or highly formal V-model	Agile (Scrum/Kanban)

Pacemaker project requires absolute predictability, exhaustive upfront documentation, and meticulous validation/verification at every phase. The V-model is ideal because every phase of development is explicitly tied to a strict testing phase, ensuring that bugs are eradicated before deployment.

For the mobile games, speed-to-market and adaptability are key. Features (levels, characters, events) change ~~wisely~~ weekly based on player engagement analytics. An agile approach allows the team to release a Minimum Viable

Date _____
Page No. _____

Product (MVP) and iteratively update ~~it~~ it based on real-world telemetry without upfront architecture overhead.

6. A customer tells you, "I know what I want. Just write a detailed contract, and I will sign it. Then build exactly what is written." Why is this approach risky in software development? Explain with a real-world example.

Ans Clients easily understand their business challenges but struggle to translate them into correct technical implementation (specifications). What they request in text often differs from what they operationalize in reality. Users find it difficult to validate functionality until they interact with an actual interface or operational prototype. Also, software cycles take months. Rigidly adhering to a frozen initial contract prevents adaptation to market or tech changes during development.

For example, let us suppose an E-commerce Platform was contracted in late 2019 with specifications strictly ~~locking~~ locking in desktop layouts and desktop web-based payment gateways. By mid-2020, the COVID-19 pandemic caused a massive surge in mobile shopping and contactless mobile wallet adoption. Because the contract was rigidly fixed, the developer delivered a flowless desktop-only platform. While legally correct, the

Date _____
Page No. _____

product was a commercial failure because it could not pivot to accommodate sudden real-world market demands.

7. Explain Brook's law: "Adding people to a late software project makes it later." Why does this happen? Give a real scenario where a manager might mistakenly violate this law.

Ans New developers are not immediately productive. Onboarding requires pulling existing developers away from active coding to mentor and explain the system, temporarily dropping net productivity.

As team size (n) expands, communication pathways increase quadratically:

$$\text{Paths} = \frac{n(n-1)}{2}$$

Adding s -

Adding staff floods the schedule with ~~syn~~ synchronization meetings and alignment emails, reducing core coding hours.

Software architecture has tight logical dependencies. Tasks cannot be subdivided indefinitely; adding labor cannot shorten an indivisible logical sequence.

A ~~co~~ For example; A corporate web portal project with a strict 6-month deadline is only 60% complete by month 5. Panicking over the delay, the manager reassigns 5 senior engineers from another project to

Date _____
Page No. _____

assist. The original developers halt coding to set up code repositories and environments for the new arrivals. Integration conflicts arise from a lack of immediate alignment, causing the project to miss the 6-month target entirely and finish in 9 months.

8. A software team is building a hospital management system. They completed requirements, design, and coding perfectly. However, when delivered, doctors refused to use it because it didn't match their work style. Which software myth caused this failure? Explain.

Ans ~~A~~ This failure was caused by a dangerous Management and customer myth.

If we compile an exhaustive set of requirements upfront and code perfectly to those specifications, the software will be successful and user adoption is guaranteed.

→ Explanation of the Failure

The team prioritized verification (building the system correctly ~~to~~ according to documents) but omitted validation (building the right system for real operational context). Doctors work in high-stress, rapidly interrupted environments. If an interface requires traversing deep nested menus to perform routine medical tasks, it represents an operational failure despite possessing "perfect code". Because the team treated requirements

as a static transactional contract instead of conducting iterative prototyping and direct workplace observations, they delivered a product that was unusable in practice.

9. Differentiate between framework activities and umbrella activities. If you are developing a simple calculator app, which umbrella activities are still necessary? Why?

Ans Framework Activities
They are core, sequential lifecycle stages required to engineer software from start to finish.

They focus on technical generation of specific engineering artifacts.

For eg: communication, planning, modeling, construction, deployment

Umbrella Activities
They are ongoing management and quality processes applied consistently across the entire lifecycle.

They focus on governance, risk mitigation and systematic quality assurance.

For eg: Software Configuration Management (SCM), Software Quality Assurance (SQA)

If I am developing a simple calculator app, then umbrella activities like SCM and SQA are still necessary as without SCM and SQA, code modifications risk destabilizing core features without a safety net.

10. The waterfall model is often criticized as being unrealistic. Yet it is ~~the~~ still used in certain projects. Give ~~me~~ two scenarios where waterfall is the right choice and two ~~se~~ scenario where it would be disastrous.

Ans Two scenarios where waterfall model is the right choice are:

- 1) Embedded Medical Systems (Safety-Critical)
Medical systems demand absolute predictability, exhaustive upfront validation, and strict regulatory documentation. Experimental iterations are too dangerous when human lives are at risk.
- 2) Fixed legacy system Migrations
Porting a mature database engine where inputs, schemas, and operational business rules are 100% stable and explicitly bounded from day one.

Ans Two scenarios where waterfall model ~~is~~ would be disastrous are:

- 1) Early-Stage Consumer Startups
Startups operate in extreme market uncertainty. Spending months on upfront planning without user validation risks building a product no one wants, exhausting capital before product-market fit.

Date _____
Page No. _____

2) Machine Learning and Data Science (R & D)
AI models depend heavily on experimental exploratory phases. It is impossible to construct a linear plan before analyzing how data behaves during initial training loops.

11. A startup wants to build a new social media app. Requirements keep changing every week. They have a budget for only 3 months. Which process model would you recommend? Justify your choice. Also explain which model would be the worst choice and why?

Ans The best model that I would recommend is Agile (Scrum). Scrum accommodates ~~vol~~ volatility. It processes developments in short 1-2 week iterations (sprints). This allows the startup to pivot requirements dynamically between sprint cycles without disrupting core development. Also scrum provides time-boxed delivery that forces strong backlog prioritization. This ensures the startup delivers MVP within their hard 3-month budget runway, avoiding an incomplete deployment.

Ans The worst choice is waterfall model, because of

- Extreme Rigidity

- Delayed Risk Prioritization
- Running out of capital at month 3 leaves the company bankrupt with zero working software to show stakeholders.

12. The spiral model is called a "risk-driven" model. What does this mean? If a project has no identifiable risks, should you still use the spiral model? Why or why not?

Ans This means the progression of the project through its evolutionary loops is directed by identifying, analyzing and mitigating potential hazards (e.g. cost inflation). Before advancing phases, the model mandates risk analysis and prototyping to resolve threats before scaling capital expenditures.

Ans No, one should avoid the spiral model if a project lacks clear risks because:

1) Excessive Administrative Overhead

The spiral model is inherently complex, requiring continuous risk assessments, audits, documentation, and specialized review cycles.

2) Resource Misalignment

For predictable, well-understood software tasks (like standard CRUD apps), the spiral model introduces immense administrative delays and inflates engineering costs for zero technical benefit. Here, a simple linear or incremental framework is far more cost-effective.

Date _____
Page No. _____

13. "Agile means no documentation and no planning." Is this statement correct or a myth? Explain the correct understanding of documentation and planning in Agile.

Ans This statement is an industry myth rooted in misinterpreting the Agile Manifesto pillar: "Working software over comprehensive documentation." It implies a shift in priority, not total elimination.

* Documentation in Agile

Agile utilizes Just-in-Time (JIT) and lean documentation. Instead of producing voluminous upfront functional specs that become obsolete immediately. Agile documents stable, high-value assets - such as APIs, architectural decisions, clean self documenting code and user stories, incrementally alongside feature extraction.

* Planning in Agile

Agile features more frequent, continuous planning than traditional methods, operating across adaptive layers.

→ Release Planning

Establishes high-level product trajectory and objectives.

→ Sprint Planning

Fine-grained dedication of deliverables

for the upcoming 1-2 weeks.

→ Daily stand-ups

Micro-planning every 24 hours to address structural blockers.

14. A client says, "I want to use Scrum because everyone is using it. But I cannot attend daily stand-up meetings because I am too busy. Also, I don't want to see the product until it is completely finished." Will Scrum work for this client? Identify the Scrum principles that will be violated.

Ans No, Scrum will fail catastrophically. The client is attempting to overlay Scrum terminology onto an unyielding waterfall mindset.

Ans The Scrum principles that will be violated are:

(a) Customer Collaboration over Contract Negotiation
Scrum requires active, ongoing alignment to continuously prioritize backlog items and clarify ambiguous requirements.

(b) Empirical Process Control (Transparency & Inspection)

Scrum depends on structural transparency through regular Sprint Reviews. Refusing to evaluate incremental software builds leaves the development team working blindly without a feedback loop.

Date _____
Page No. _____

© Incremental Value Delivery:

Scrum functions by deploying usable, vertical software Increments every 2 to 4 weeks. Waiting for a final monolithic delivery invalidates the core agility of the framework.

15. You are the technical lead for a project with the following characteristics:
- i. Requirements are mostly clear but some features are uncertain.
 - ii. Customer is available only once a month.
 - iii. System is safety-critical (failure could cause injury)
 - iv. Team size is 20 people
 - v. Deadline is fixed and cannot be changed.
- Which process model would you choose? Justify your choice by comparing at least two possible models against the given constraints.

Ans The Incremental V-Model is the best choice because it combines the rigorous testing and verification required for a safety-critical system with the flexibility needed to handle uncertain requirements.

Constraint	Agile (Fails)	Waterfall (Fails)	Incremental (Works) ✓
Safety-Critical	Insufficient formal verification	Testing occurs too late	Provides strict validation & verification at every step.
Uncertain Feature	Risk of scope creep	Requires early fixed requirements	Allows feature refinements through increments.
Monthly Customer Access	Needs frequent feedback	Customer sees product very late	Fits monthly review cycles.
Fixed Deadline	Scope may expand	Late defects can delay delivery	Delivers core features first, reducing schedule risk.
Team Size (20)	Too large for one Scrum team	Can create communication silos	Supports multiple specialized teams in parallel.

Hence, the Incremental-V model offers the safety assurance of the V-model while allowing controlled adaptation to changing requirements making it the most suitable approach for this project.